第10章作业预告：

10.1　10.6　10.8　10.25　10.42

第10章 内部排序

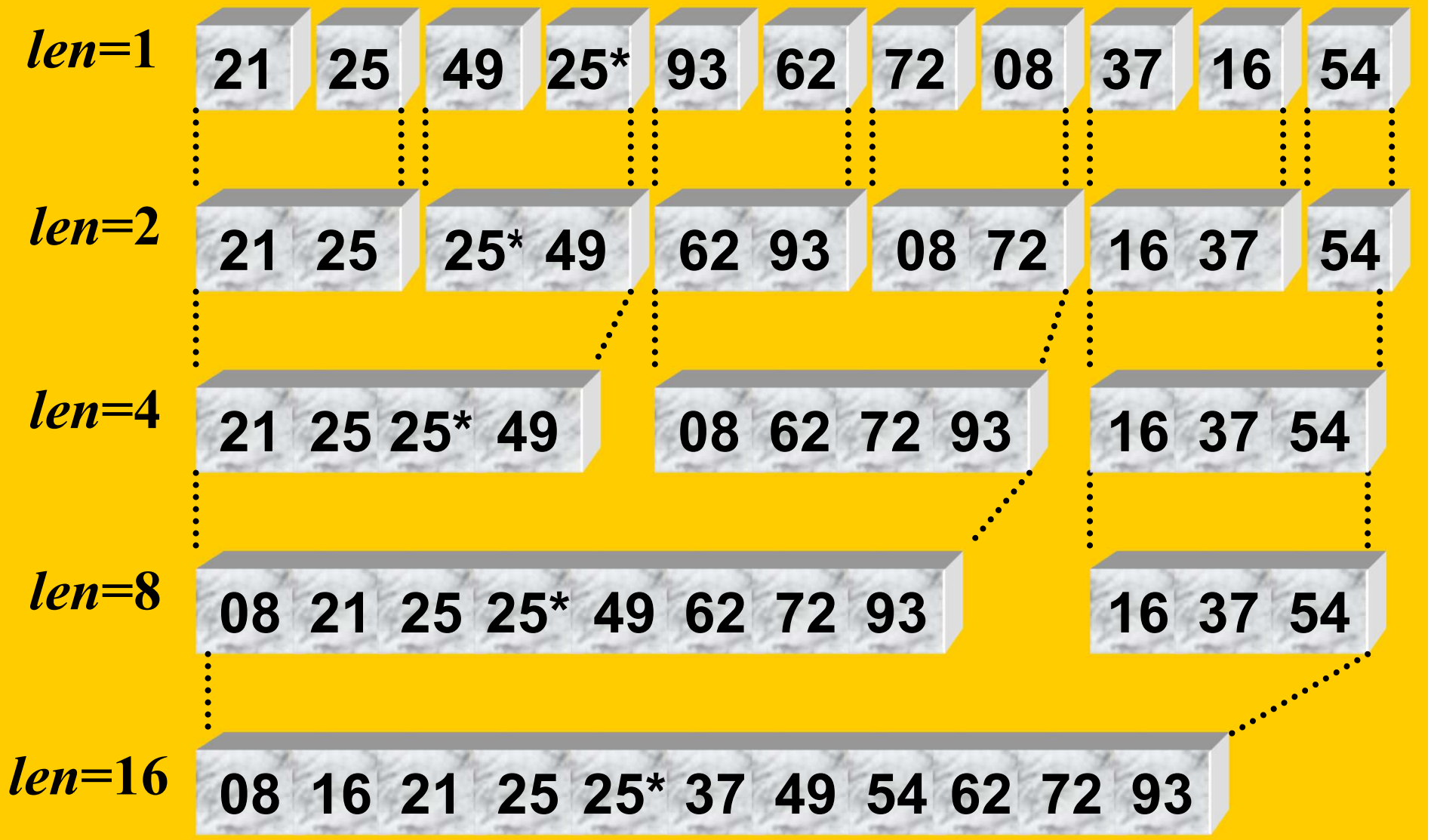
10.1 概述
10.2 插入排序
10.3 交换排序
10.4 选择排序
10.5 归并排序
10.6 基数排序

10.5 归并排序

归并排序的基本思想是：将两个（或以上）的有序表组成新的有序表。

更实际的意义：可以把一个长度为n 的无序序列看成是 n 个长度为 1 的有序子序列，首先做两两归并，得到$\lceil n/2 \rceil$个长度为 2 的有序子序列；再做两两归并，…，如此重复，直到最后得到一个长度为 n 的有序序列。

例：关键字序列T=（21，25，49，25*，93，62，72，08，37，16，54），请给出归并排序的具体实现过程。



整个归并排序仅需 $\lceil \log_2 n \rceil$ *趟，但*
每趟仍要比较和整理n个元素

一趟归并排序算法: (两路有序并为一路)　参见教材P283

```
void Merge (SR，&TR，i, m, n) {
    // 将有序的SR[i…m]和SR[m+1…n]归并为有序的TR[i…n]
  for(k=i , j=m+1;  i<=m && j<=n;  ++k ) {
        if ( SR[i]<= SR[j] )TR[k]=SR[i++];
        else TR[k]=SR[j++]; // 将两个SR记录由小到大并入TR
 } // for
  if (i<=m) TR[k…n]=SR[i…m]; // 将剩余的SR[i…m]复制到TR
  if (j<=n) TR[k…n]=SR[j…n]; // 将剩余的SR[j…n]复制到TR
}  // Merge
```

这就是能提高排序速度的原因

递归形式的两路归并完整排序算法： 参见教材P284

```
void MSort (SR[ ],&TR1[ ]，s, t) {
                    // 将无序的SR[s…t]归并排序为TR1[s…t]
```

初次调用时为（L, L, 1, length）

```
if ( s==t )TR1[s]=SR[s];        // 当1 = length时返回
   else {
      m=(s+t)/2;   // 将SR [s…t]平分为SR [s…m]和SR [m+1…t]
      MSort (SR，&TR2，s, m);    // 将SR 一分为二, 2分为4…
                    // 递归地将SR [s…m]归并为有序的TR2[s…m]
      MSort (SR，&TR2，m+1, t );
                    // 递归地将SR [m+1…t]归并为有序的TR2[m+1…t]
      Merge(TR2，  TR1，  s, m, t );
                    // 将TR2 [s…m]和TR2 [m+1…t]归并到TR1 [s…t]
  } //if
} // MSort
```

简言之，先由“长”无序变成“短”有序，
再从“短”有序归并为“长”有序。

归并排序算法分析：

- 时间效率： $O(n\log_2 n)$

在每趟归并排序的操作中，要调用$\lceil n/(2h) \rceil$次*Merge*( )算法，将SR[1…n]中前后相邻且长度为h的有序段进行两两归并，得到前后相邻、长度为2h的有序段，并存放在TR [1…n]中。另外，整个归并排序有$\lceil \log_2 n \rceil$“层”，所以算法总的时间复杂度为$O(n\log_2 n)$。

- 空间效率： $O(n)$

因为需要一个与原始序列同样大小的辅助序列（TR）。这正是此算法的缺点。

- 稳定性：稳定

10.6 基数排序 (Radix Sort)

基数排序的基本思想是：

借助多关键字排序的思想对单逻辑关键字进行排序。即：用关键字不同的位值进行排序。

要讨论的问题：

1. 什么是“多关键字”排序？实现方法？

2. 单逻辑关键字怎样“按位值”排序？

例1：对一副扑克牌该如何排序?

若规定花色和面值的顺序关系为：

花色： ♦ < ♣ < ♥ < ♠

面值： 2 < 3 < 4 < 5 < 6 < 7 < 8 < 9 < 10 < J < Q < K < A

则可以先按花色排序，花色相同者再按面值排序；

也可以先按面值排序，面值相同者再按花色排序。

最高位优先法**MSD**
先设立4个花色“箱”，将全部牌按花色分别归入4个箱内（每个箱中有13张牌）；然后对每个箱中的牌按面值进行插入排序（或其它稳定算法）。

最低位优先法**LSD**
先按面值分成13堆（每堆4张牌），然后对每堆中的牌按花色进行排序（用插入排序等稳定的算法）

讨论：是借用MSD方式来排序好呢，还是借用LSD方式?

例：初始关键字序列T=（322，135，274，322*，131，336，147），请分别用MSD和LSD进行排序，并讨论其优缺点。

法1（MSD）：原始序列：322，135，274，322*，131，336，147

先按高位K_i^1排序：(135，131，147)，274，(322，322*，336)

再按低位K_i^2排序：((135，131)，147)，274，((322，322*)，336)

因为有分组，故此算法需递归实现。

法2（LSD）：原始序列：322，135，274，322*，131，336，147

先按低位K_i^3排序：131，322，322*，274，135，336，147

再按中位K_i^2排序：322，322*，131，135，336，147，274

最后按高位K_i^1排序：131，135，147，274，322，322*，336

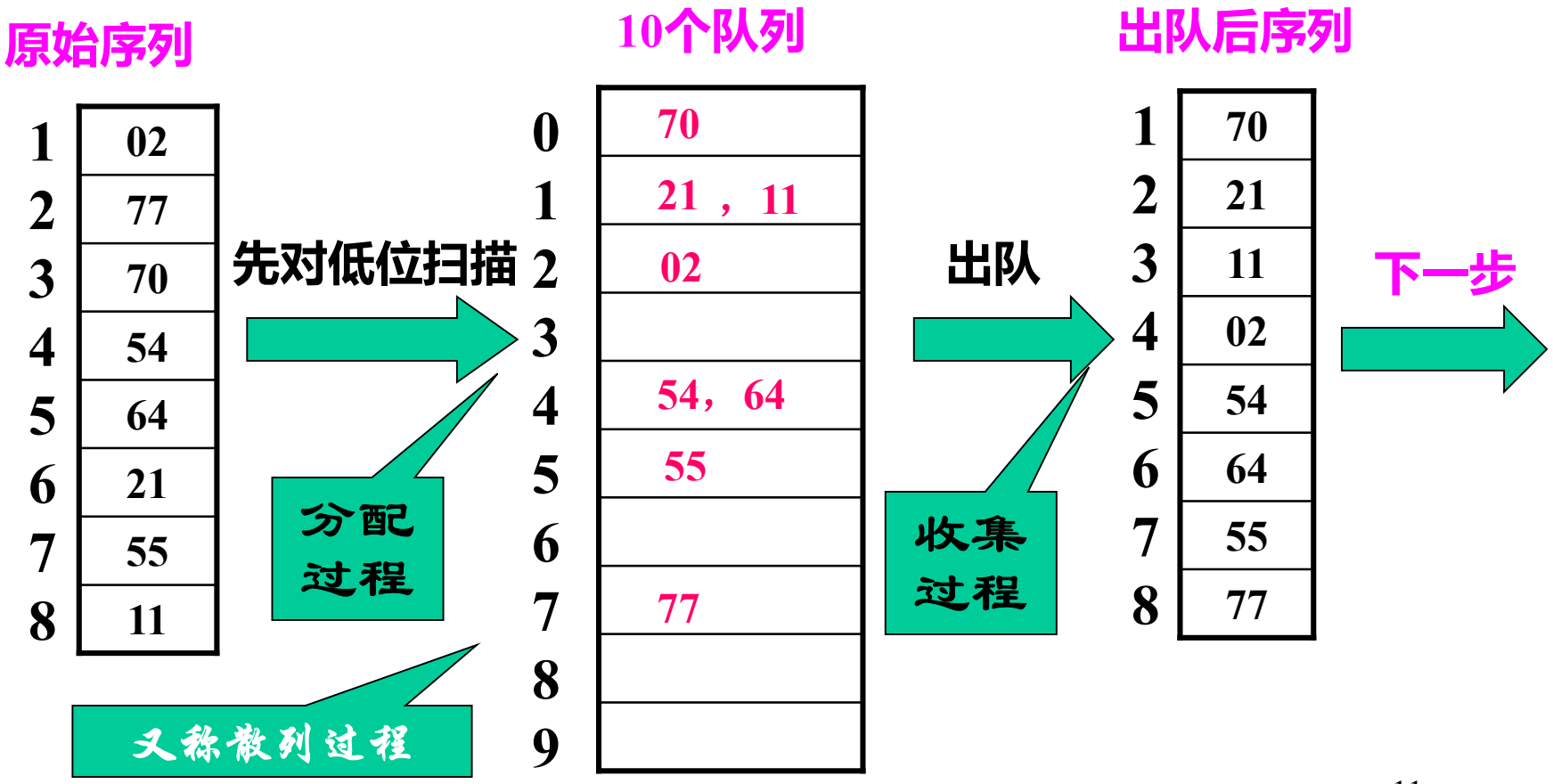
无需分组，易编程实现！

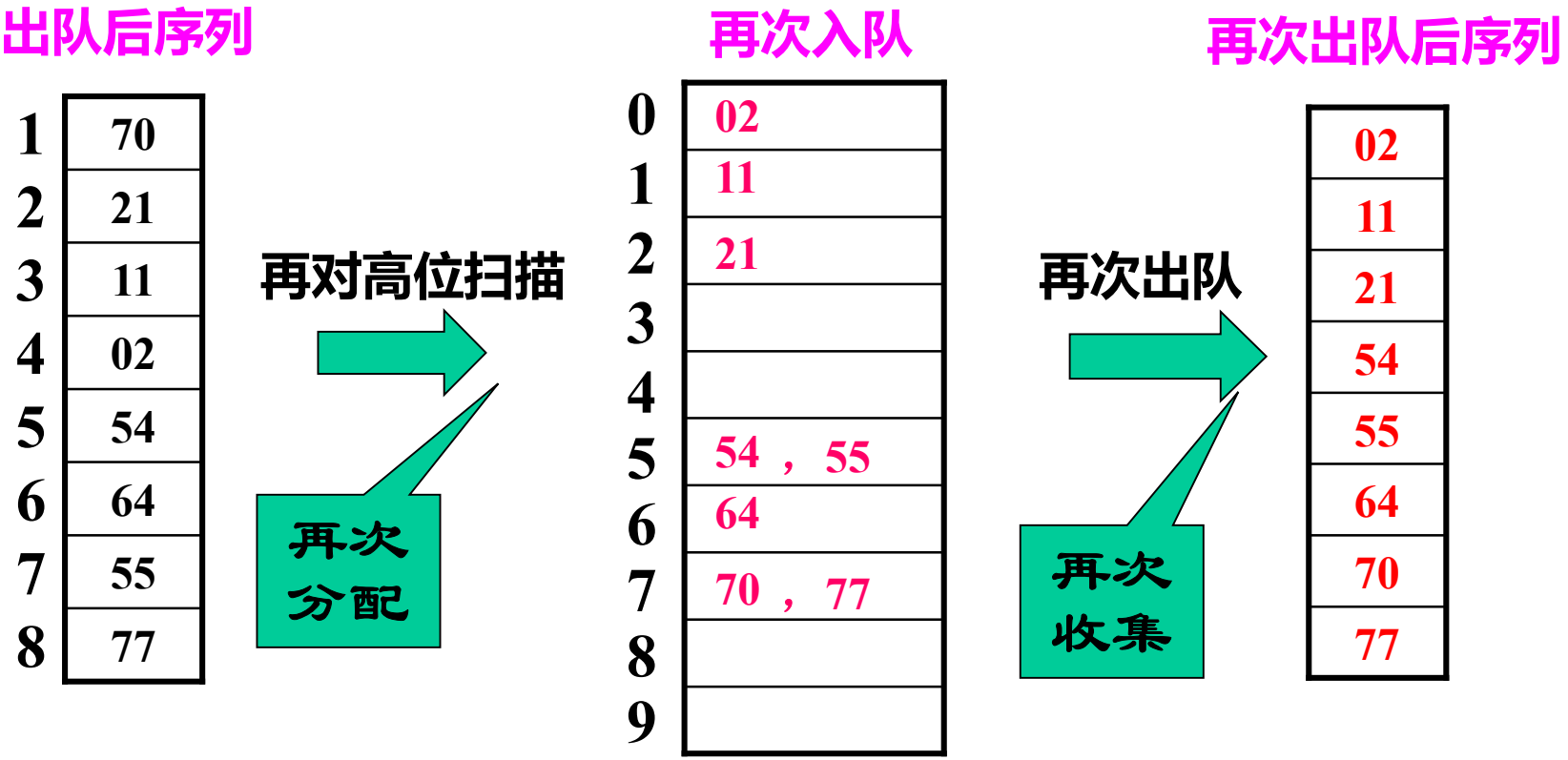
计算机怎样实现LSD算法？

例：T=（02，77，70，54，64，21，55，11），用LSD排序。

分析：

①各关键字可视为2元组；②每位的取值范围是：0-9；即基数 *radix* = 10 。因此，特设置10个队列，并编号为0-9。





小结：排序时经过了反复的“分配”和“收集”过程。当对关键字所有的位进行扫描排序后，整个序列便从无序变为有序了。

这种LSD排序方法称为：基数排序

讨论：所用队列是顺序结构，浪费空间，能否改用链式结构？

能！ **用链队列来实现基数排序——** 链式基数排序

实现思路：针对 d 元组中的每一位分量，把原始链表中的所有记录，按 K_j 的取值，让 $j = d, d-1, \ldots, 1$，

① 先“分配”到 $radix$ 个链队列中去；

② 然后再按各链队列的顺序，依次把记录从链队列中“收集”起来；

③ 分别用这种“分配”、“收集”的运算逐趟进行排序；

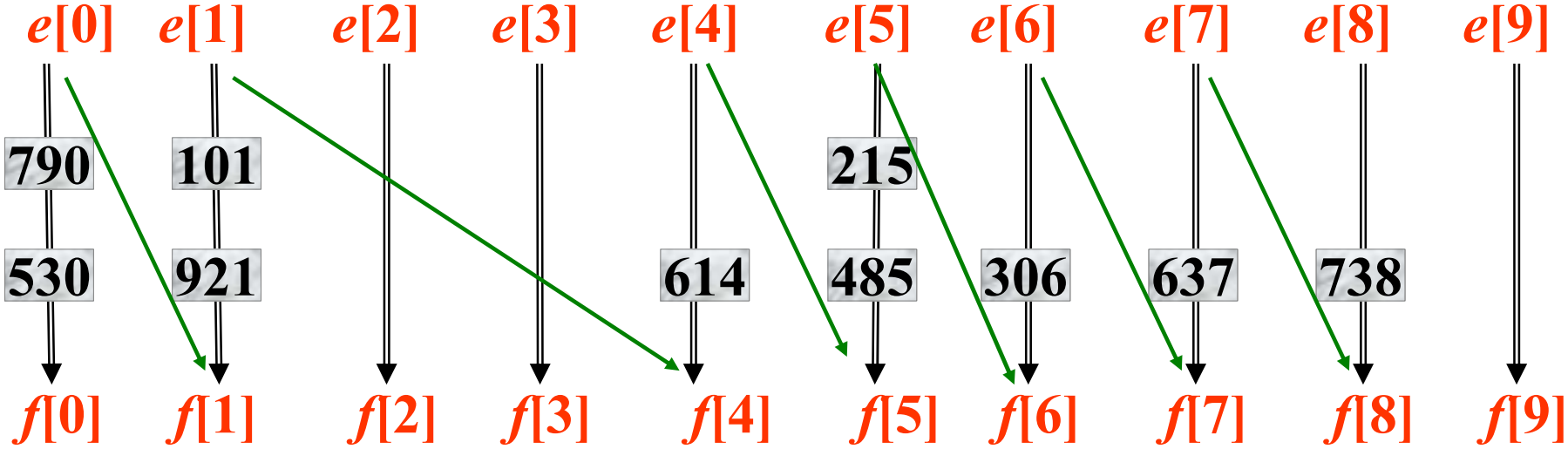
④ 在最后一趟“分配”、“收集”完成后，所有记录就按其关键码的值从小到大排好序了。

例:请实现以下关键字序列的链式基数排序:

T= (614, 738, 921, 485, 637, 101, 215, 530, 790, 306)

原始序列静态链表:

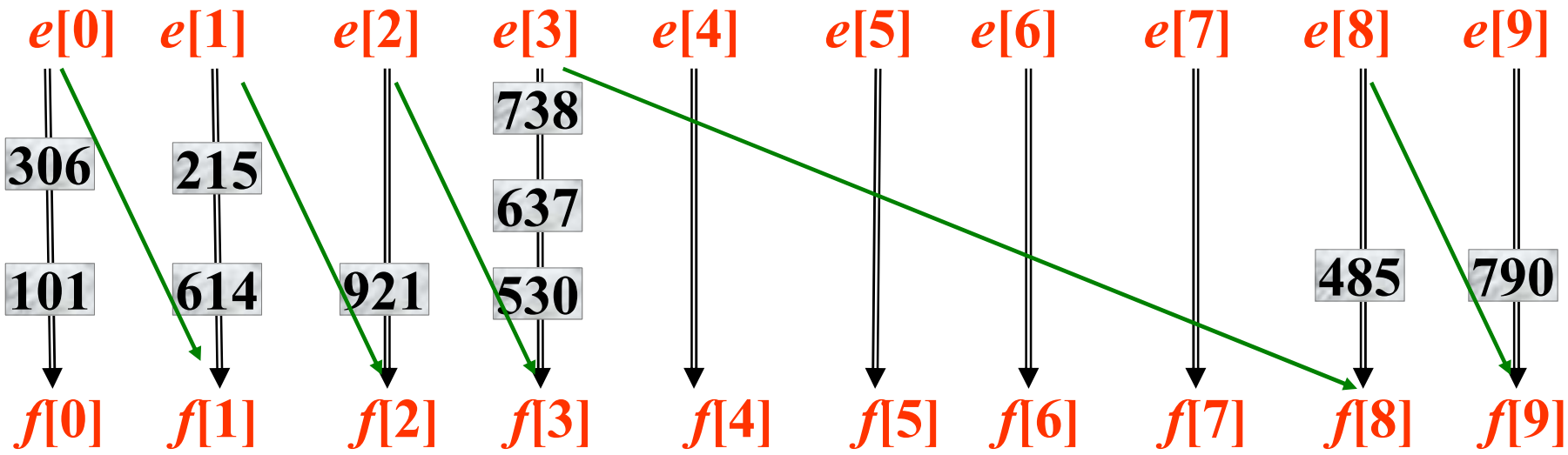
第一趟分配 (从最低位 $i = 3$开始排序, f[ ] 是队首指针, e[ ] 为队尾指针)



第一趟收集 (让队尾指针e[i] 链接到下一非空队首指针f[i+1] 即可)

第一趟收集的结果：

第二趟分配（按次低位 $i = 2$ ）

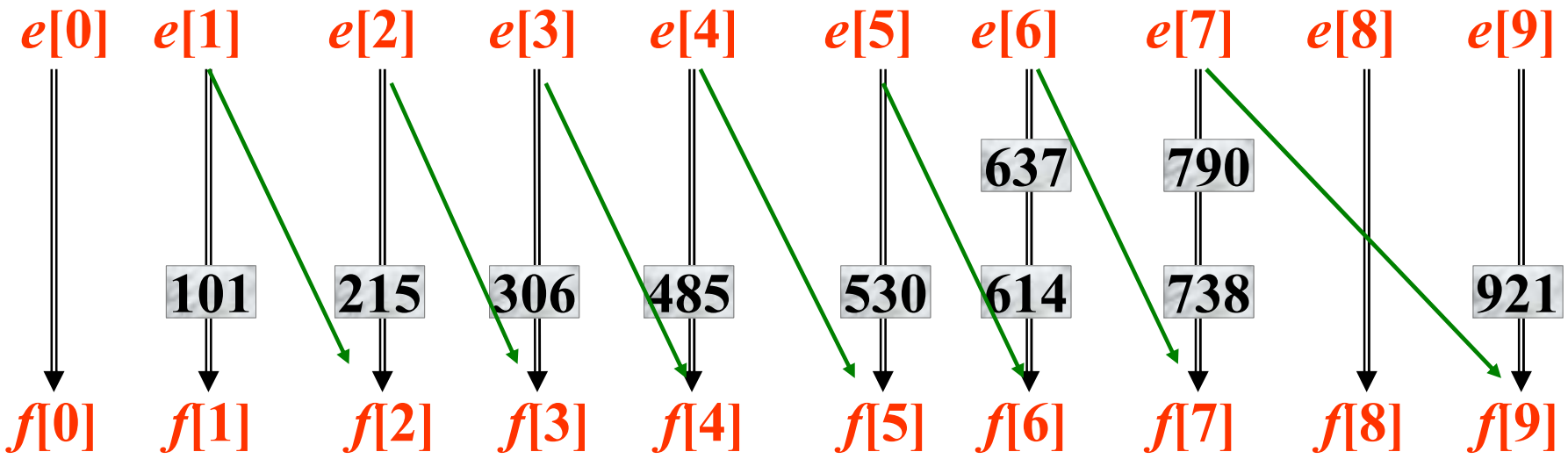


第二趟收集（让队尾指针e[i] 链接到下一非空队首指针f[i+1] ）

第二趟收集的结果：

r[0]→ 101 → 306 → 614 → 215 → 921 → 530 → 637 → 738 → 485 → 790

第三趟分配（按最高位 $i = 1$ ）



第三趟收集（让队尾指针e[i] 链接到下一非空队首指针f[i+1] ）

r[0]→ 101 → 215 → 306 → 485 → 530 → 614 → 637 → 738 → 790 → 921

排序结束！

基数排序算法分析

特点： 不用比较和移动，改用分配和收集，时间效率高！

- 假设有n 个记录, 每个记录的关键字有d 位，每个关键字的取值有$radix$个，则每趟分配需要的时间为O(n)，每趟收集需要的时间为O($radix$)，合计每趟总时间为O ($n+radix$)

  全部排序需要重复进行d 趟“分配”与“收集”。因此时间复杂度为：O (d ($n+radix$))。
- 基数排序需要增加$n+2radix$个附加链接指针，空间效率低

  空间复杂度：O(n+$radix$) .
- 稳定性：稳定。(一直前后有序)。

用途： 若基数$radix$相同，对于记录个数较多而关键码位数较少的情况，使用链式基数排序较好。

如何编程实现？ **先作若干说明：**

1.建议用**静态链表**形式存储排序前后的n个记录（在r[ ]中增开n个指针分量）；这样在记录重排时不必移动数据，只需修改各记录的链接指针即可。

2.在*radix个静态队列中*，每个队列都要设置两 个指针：

int *f* [*radix*]指示队头(f[j]初始为空）；

int *e* [*radix*] 指向队尾（e[j]可省略初始化,队尾元素随时赋值)；

分配到同一队列的关键码要用链接指针链接起来。

（注：以上一共增开了n+2 *radix个附加指针分量*）

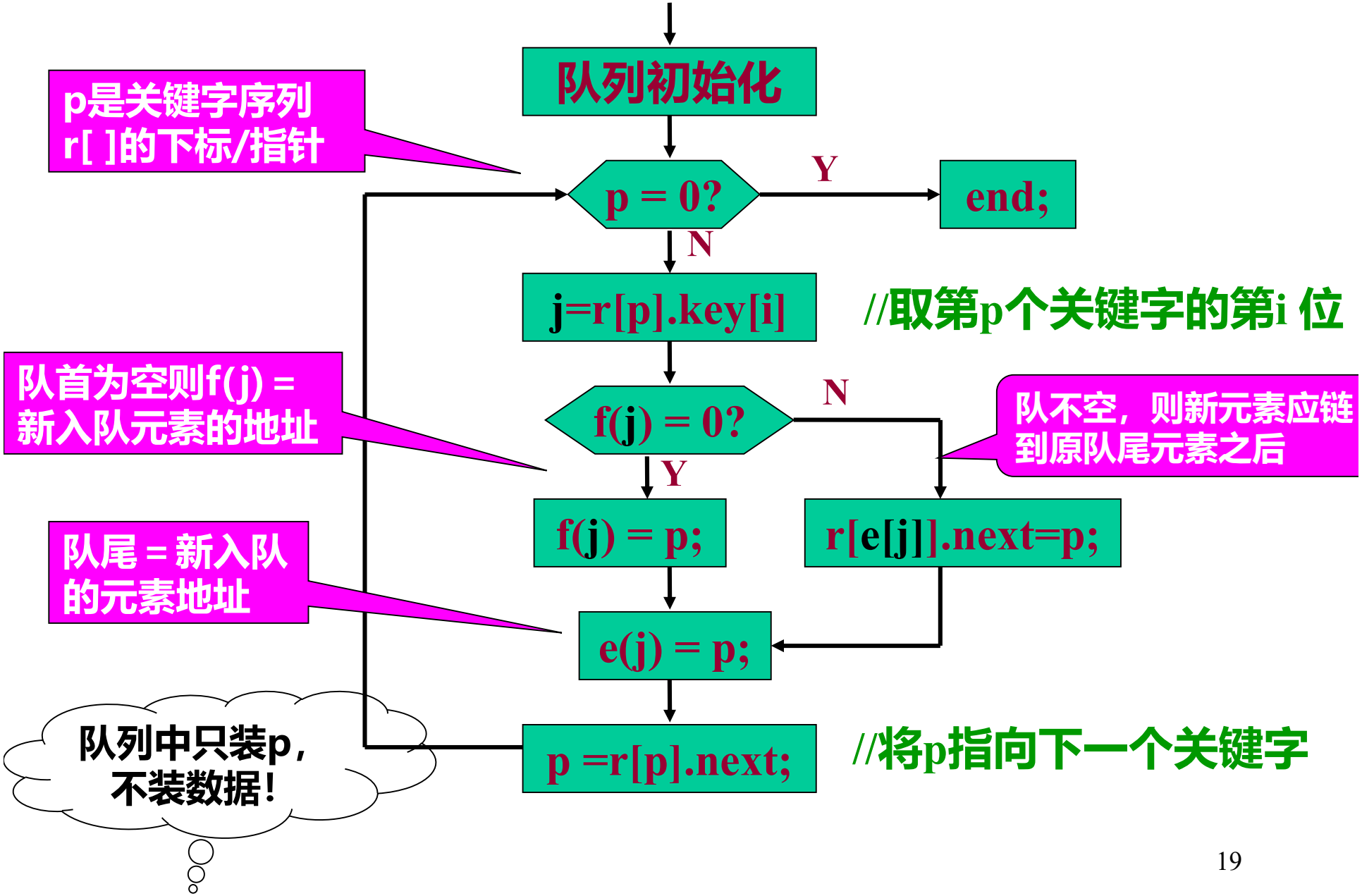
3. 存入r[ ]中的待排记录可分为3部分：

key [0…d-1]	othernifo	next

第p个记录的关键码的第i位可表示为： r[p].key[i]

第p个记录的指针分量可表示为： r[p].next

一趟“分配”过程的算法流程



0链表基数排序的算法：

```
void RadixSort (&list, int d,  int radix ) {
   int f[radix], e[radix];
   for ( int i = 1;  i < n;  i++ )  r[i].next=i+1;
    r[n].next=0;            //静态链表初始化，将各记录顺次链接。
    int p= 1;               //p是链表元素的地址指针
   for ( i = d-1; i >= 0; i-- ) {  //开始做 d 趟分配/收集, i是key的第i位
                                   //LSD算法规定先从后面开始排序
      for ( int j = 0; j < radix; j++ )  f[j] = 0;   //初态 = 各队列清空
      while ( p!= 0 ) {      // 开始将n个记录分配到radix个队列
        int k = r[p]. key[i];       //取当前记录之key分量的第 i 位
       if ( f[k] = = 0) f [k] =p;  //若第k个队列空, 此记录成为队首;
         else r[e[k]].next=p;  //若队列不空,链入原队尾元素之后
           e[k] =p;              //修改队尾指针，该记录成为新的队尾
         p= r[p].next; } //while 选下一关键字，直到本趟分配完
```

```
        j = 0;   // 开始从0号队列（总共radix个队）开始收集
      while ( f [j] = = 0 ) j++;     //若是空队列则跳过
      r[0].next=p = f [j];          //建立本趟收集链表的头指针
      int last = e[j];              //建立本趟收集链表的尾指针
      for ( k = j+1; k < radix; k++)   //逐个队列链接（收集）
        if ( f [k] ≠0) {                //若队列非空
            r[last].next=f [k];  last = e[k];  //队尾指针链接
        } // for
      r[last].next=0;          //本趟收集链表之尾部应为0
    } // for
} // RadixSort
```

注：在此算法中，数组r[n]被反复使用，用来存放原始序列和每趟收集的结果。记录从未移动，仅指针改变。

各种内部排序方法的比较 (教材P289）

排序方法	最好情况	平均时间	最坏情况	辅助存储	稳定性
简单排序	O(n)	$O(n^2)$	$O(n^2)$	O(1）	稳定
快速排序	O(nlgn)	O(nlgn)	$O(n^2)$	O(lgn)	不稳定
堆排序	O(nlgn)	O(nlgn)	O(nlgn)	O(1）	不稳定
归并排序	O(nlgn)	O(nlgn)	O(nlgn)	O(n）	稳定
基数排序	O(d(n+rd))	O(d(n+rd))	O(d(n+rd))	O(rd)	稳定
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	O（1）	不稳定
直接插入	O(n)	$O(n^2)$	$O(n^2)$	O（ 1）	稳定
折半插入	O(nlgn)	$O(n^2)$	$O(n^2)$	O（ 1）	稳定
冒泡	O(n)	$O(n^2)$	$O(n^2)$	O（ 1）	稳定